SIH 26164

ECDAT

Enterprise Cryptographic Discovery and Quantum Risk Analysis Tool

Solution architecture, detection catalogue, risk engine, PQC recommender, CLI and GUI

Optimized for high coverage, low false positives, scalable scanning and judge-grade output evidence

Reference document - September 2026

1. Executive decision

Build **ECDAT as a modular discovery platform**, not as a simple regex scanner. The defensible product is a **confidence-aware cryptographic inventory and risk graph** that can accept one input or many inputs, reconstruct crypto operations, output a standardized CBOM, and recommend post-quantum or hybrid migration paths.

Core decision: **do not use LLVM or MLIR**. Their ROI is weak for a mixed enterprise estate containing Java, Python, JavaScript, Go, C#, C/C++, Rust, binaries, containers and cloud services. Use a domain-specific **Crypto Intermediate Representation (CIR)** plus a **Crypto Knowledge Graph** instead. This keeps high-level crypto semantics instead of lowering them into compiler IR.

Area	Decision	Reason
Mandatory capability	Source, binary, library and container scanning	The expected solution explicitly asks for these scan surfaces.
Broader PS artefacts	Certificates, keys, protocols, hardware modules, cloud KMS/services	The description asks to identify cryptographic artefacts across apps, products and infrastructure.
Main innovation	Operation reconstruction	Do not merely list RSA/AES strings; reconstruct crypto operations and systems with evidence.
Winning proof	Coverage + precision + benchmarked speed	Compare against CBOMkit/cdxgen/source grep/binary strings and show fewer false positives plus more linked artefacts.

2. Scope: required vs innovative scan surfaces

ECDAT should work even if the user provides only one input. A repo-only scan should produce useful results. A repo + container + runtime + cloud scan should produce stronger evidence and higher confidence.

Surface	User input examples	PS status	Tag	Primary method
Source repositories	Local folder, Git URL, uploaded zip	Expected	Required	CBOMkit/Sonar where strong; Semgrep rules; Tree-sitter AST; limited dataflow on candidates.
Binaries	ELF, PE, Mach-O, shared libraries, JAR/APK	Expected	Required	LIEF imports/symbols; YARA-X constants; capa behavior rules; FLOSS strings; Ghidra only if needed.
Libraries/dependencies	pom.xml, package-lock, Cargo.lock, requirements.txt, OS packages	Expected	Required	Syft/cdxgen plus package-manager parsers; map packages to crypto capability registry.
Container images	Docker/OCI image, image tar, rootfs directory	Expected	Required	Layer extraction + cdxgen/Syft/Trivy + inner source/binary/config/cert scanners.
Config files	nginx, Apache, Envoy, OpenSSL config, YAML, TOML, Terraform, Kubernetes	Supported by description	Innovative	Generic config parser + Semgrep generic rules + service-specific TLS/cipher suite parser.
Certificates and keys	PEM, DER, JKS, PKCS#12, SSH keys, JWKs	Supported by description	Innovative	X.509/ASN.1 parsers + Gitleaks/TruffleHog; redact material; store fingerprint and metadata only.
SBOM/CBOM import	CycloneDX JSON/XML, SPDX	Not explicit but useful	Innovative	Ingest existing BOMs; enrich them with missing crypto evidence and risk analysis.
Runtime endpoints	HTTPS, SSH, STARTTLS, internal service URLs	Infrastructure oriented	Innovative	SSLyze/testssl.sh style probes; compare negotiated crypto with static config.
Cloud services	AWS KMS/ACM/ELB, Azure Key Vault, GCP KMS, HSM metadata	Description mentions cloud services	Innovative	Read-only cloud inventory connectors; no secret extraction; map keys/certs/services to graph.

Input rule: every detector is optional. The graph stores missing evidence explicitly. Confidence rises when multiple surfaces agree.

3. Optimized architecture

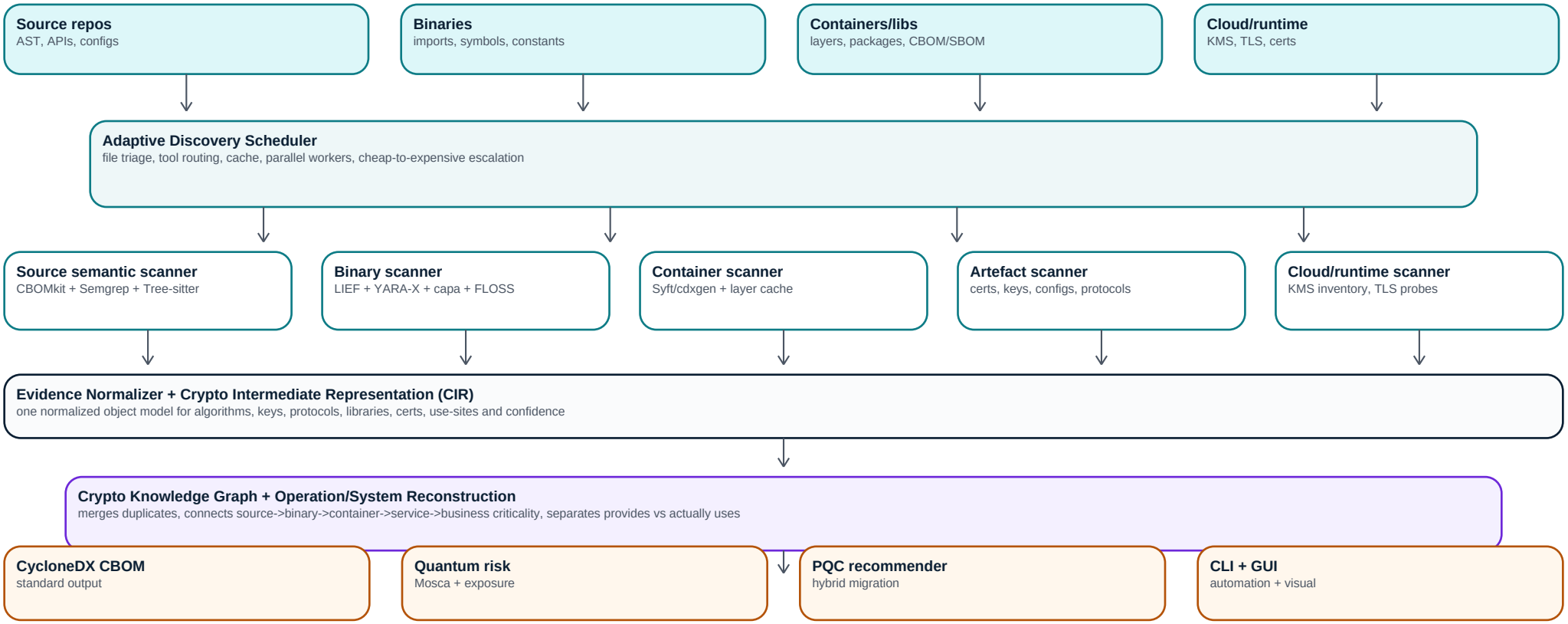


Figure 1. ECDAT architecture. The product is not a wrapper around one scanner. It is a staged evidence system that routes each artefact to the cheapest reliable detector first, escalates only when needed, and merges evidence into a single graph.

3.1 Core engines

Engine	Build/reuse	What it does	Efficiency rule
Input registry	Build	Normalizes repo, image, binary, directory, URL and cloud targets into scan jobs.	Hash every target; skip unchanged artefacts.
Adaptive scheduler	Build	Routes files by MIME/type/extension/size; picks cheap detector first and escalates.	No Ghidra/capa/dataflow until cheap signals say it is worth it.
Signature registry	Build	Canonical names, aliases, OIDs, library APIs, weak modes, PQC mappings.	Registry-driven additions instead of code changes.
Source scanner	Reuse + build	CBOMkit/Sonar + Semgrep + Tree-sitter + custom CIR emitter.	AST/dataflow only on crypto candidate slices.
Binary scanner	Reuse + build	LIEF, strings, YARA-X, capa/FLOSS, optional Ghidra.	Imports/symbols -> strings -> constants -> behavior -> decompiler.

Engine	Build/reuse	What it does	Efficiency rule
Container scanner	Reuse + build	Layer cache, Syft/cdxgen/Trivy, then inner source/binary/config/cert passes.	Cache by image/layer digest.
CIR builder	Build	Converts all detector outputs into CryptoOperation/CryptoAsset objects.	One downstream pipeline for every language and artefact.
Graph builder	Build	Resolves duplicates and links operations, keys, certs, containers, services and owners.	Deduplicate before risk scoring.
Risk engine	Build	Quantum vulnerability + data lifetime + migration time + exposure + business criticality.	Score only graph-resolved systems, not every raw hit.
Recommender	Build	Suggests PQC/hybrid alternatives with compatibility and performance notes.	Rule-based first; LLM only for explanation, not detection.
Export layer	Build	CycloneDX CBOM, JSON, SARIF, PDF/HTML report, dashboard API.	Canonical JSON is source of truth.

4. Existing tools: where to use them and where to build our own

Use existing tools for solved low-level scanning and parsing. Build the parts that competitors usually lack: evidence fusion, CIR, operation reconstruction, graph context, risk scoring and recommendations.

Need	Best tool choice	Why not build from scratch	Our added value
CBOM format	CycloneDX CBOM	It already models cryptographic assets and dependencies.	Emit clean CBOM, add confidence/evidence extensions, import and enrich existing BOMs.
Source crypto APIs	CBOMkit/Sonar + Semgrep	Known Java/Python/Go/.NET patterns are already captured well.	Add JS/TS, Node crypto, WebCrypto, RustCrypto, C/C++ OpenSSL/libsodium wrappers and dataflow.
AST parsing	Tree-sitter	Fast multi-language syntax trees.	Convert matches into CIR and graph relations.
Deep source reasoning	CPG-inspired graph; optional Joern for research	CPG merges syntax, control flow and dataflow ideas without compiler IR lock-in.	Use only on candidates to keep scan time low.
Dependency inventory	Syft + cdxgen	They already understand many package ecosystems and container filesystems.	Map package versions to crypto capability and vulnerability/risk metadata.
Containers	cdxgen + Syft + Trivy + CBOMkit-theia ideas	They already inspect images, rootfs and packages.	Layer-digest cache plus cross-link inner evidence to image/service.
Executable metadata	LIEF	Cross-platform parser for ELF/PE/Mach-O.	Convert imports, linked libs and symbols into binary evidence.
Binary signatures	YARA-X	High-scale binary/text pattern engine.	Crypto constants/signature rules for AES/SHA/ChaCha/MD5/PQC libraries.
Binary behavior	capa	Capability rules work well for compiled executables.	Custom crypto behavior namespace.
Obfuscated strings	FLOSS	Static deobfuscation of strings is hard to implement.	Run only on binaries where plain strings are weak.
Deep reverse engineering	Ghidra	Strong decompiler/disassembler but too heavy for all files.	Escalation-only backend for high-value ambiguous binaries.
Secrets/keys	Gitleaks + TruffleHog	Mature secret/key detectors.	Never store raw secrets; store fingerprint, type, size, path and redacted evidence.
Live TLS	SSLyze or testssl.sh	Already enumerates protocols, ciphers, certs and TLS weakness signals.	Join runtime evidence with source/config/container graph.
PQC implementation catalogue	Open Quantum Safe / liboqs	Useful for naming and migration compatibility references.	Use as recommendation/experiment reference, not as detector core.

Build vs reuse principle: if an existing scanner is better at raw extraction, use it. If the task is correlation, confidence, graph context, quantum prioritization or PQC migration advice, build it.

5. Crypto Intermediate Representation (CIR)

CIR is the internal semantic model. It gives the architectural benefits people usually chase with compiler IR, but it is built for cryptography and works across source, binaries, containers, configs, certificates and cloud assets.

```
CryptoOperation {
  id, operation_type: Encrypt | Decrypt | Sign | Verify | KeyExchange | Hash | KDF | TLS | Storage
  algorithm: canonical_name, family, variant, mode, padding, key_size, curve, parameters
  material: key_id, cert_id, keystore_id, kms_key_id, nonce_or_iv, rng_source
  implementation: library, version, provider, API, binary_symbol, package, container_layer
  location: file, line, function, binary_offset, image_digest, service, endpoint
  evidence: [detector, signal_type, confidence, raw_snippet_redacted]
  context: exposure, data_type, data_lifetime, business_criticality, owner, environment
}
```

CIR objects then become graph nodes. The graph should explicitly distinguish "provided by a library" from "actually used by the application". This avoids noisy CBOMs that list every algorithm inside OpenSSL as if the app used all of them.

Evidence signal	Example	Confidence impact
Direct source API call	Cipher.getInstance("AES/GCM/NoPadding")	High: direct usage and mode are visible.
Resolved constant/dataflow	mode variable resolves to "GCM"	Raises confidence and extracts parameters.
Dependency provides capability	OpenSSL linked in package list	Low alone: capability, not usage.
Binary import/symbol	EVP_aes_256_gcm or BCryptEncrypt	Medium to high depending on call context.
YARA constant match	AES S-box or SHA-256 constants in stripped binary	Medium: custom/static implementation evidence.
Config/cert match	nginx uses RSA cert + TLS 1.2 ciphersuite	High for deployment context.
Runtime probe	Endpoint negotiates ECDHE_RSA_WITH_AES_256_GCM	High for live exposure.
Cloud inventory	KMS key spec RSA_2048 used by production service	High for infrastructure asset, lower for code use unless linked.

6. Efficient detection pipeline

The main performance win is not one magic algorithm. It is staged scanning, target routing, caching and avoiding expensive analysis unless cheap evidence fails.

Stage	Action	Why it is efficient
0. Inventory	Walk files, image layers, packages and manifests; compute content hashes.	Creates a cache key for incremental and repeated scans.
1. Triage	Skip irrelevant files; classify source/config/binary/archive/cert/key.	Prevents every detector from touching every file.
2. Cheap source scan	Semgrep/Tree-sitter pattern rules on likely crypto files.	Fast, language-aware and much better than global regex.
3. Candidate dataflow	Resolve constants, aliases, wrappers and cross-file calls only near crypto candidates.	Coverage improves without whole-program cost.
4. Dependency scan	Syft/cdxgen lockfile and package analysis.	Avoids manually parsing every package ecosystem.
5. Binary cheap pass	LIEF imports/symbols + strings + YARA-X quick rules.	Gets most useful binary evidence without decompilation.
6. Binary expensive pass	capa/FLOSS/Ghidra only for stripped or high-value binaries.	Turns heavy reverse engineering into an escalation path.
7. Container pass	Reuse package/layer digest cache; run inner detectors on changed layers only.	Massive speedup in CI and repeated image scans.
8. Evidence merge	Deduplicate aliases, repeated files, vendored libraries and source/binary duplicates.	Cuts false positives and dashboard noise.
9. Risk scoring	Score resolved operations/systems, not raw findings.	A repo with 80 RSA mentions may become 3 actual risk objects.

6.1 Confidence scoring

```
finding_confidence = weighted_fusion(
  direct_usage_evidence,
  parameter_resolution,
  binary_support,
  config_runtime_agreement,
  dependency_only_penalty,
  detector_reliability,
  duplicate_correlation
)

usage_state = Confirmed | Probable | Possible | ProvidedOnly | FalsePositiveSuppressed
```

7. Detection catalogue

This is the practical coverage target. Each detector emits evidence into CIR. The table intentionally separates source/config detection from binary/container detection.

Crypto / artefact	Source or config method	Binary/container method	Risk / recommendation note
RSA	JCA/JCE, OpenSSL EVP_PKEY_RSA, pyca RSA, Node crypto RSA, .NET RSA, PEM/OID/key-size extraction.	LIEF symbols/imports, RSA strings/OIDs, ASN.1 keys/certs, OpenSSL/CNG calls, big-int exponentiation hints.	Quantum-vulnerable public key. Prefer ML-KEM for KEM use; ML-DSA/SLH-DSA for signatures; hybrid during transition.
DSA	DSA APIs, SHAwithDSA strings, BouncyCastle/JCA providers.	DSA symbols/OIDs, algorithm strings, library calls.	Legacy and quantum-vulnerable. Replace with ML-DSA or classical modern fallback until PQC ready.
DH / FFDHE	DH APIs, SSL/TLS params, group/key length.	DH_compute_key, EVP_PKEY_DH, TLS cipher suites, parameter blobs.	Quantum-vulnerable key exchange. Move to TLS 1.3 and ML-KEM hybrid where available.
ECDH	ECDH APIs, curve names, KeyAgreement("ECDH"), WebCrypto deriveBits.	EC symbols, curve OIDs/names, TLS ciphersuites, X25519/ECDH symbols.	Quantum-vulnerable. Replace/augment with ML-KEM hybrid.
ECDSA	ECDSA APIs, SHA256withECDSA, curve names in certs/config.	ECDSA symbols/OIDs, cert signature algorithm, EC library calls.	Quantum-vulnerable signatures. Plan ML-DSA or SLH-DSA migration.
X25519 / X448	libsodium/OpenSSL/WebCrypto key agreement identifiers.	Symbols/strings are strong; curve constants as fallback.	Quantum-vulnerable despite modern classical strength. Use hybrid with ML-KEM.
Ed25519 / Ed448	Signing API identifiers, SSH keys, cert experiments.	Symbols/strings/constants.	Quantum-vulnerable signatures. Move to ML-DSA/SLH-DSA when ecosystem supports.
ElGamal	GPG/BouncyCastle class names and algorithm strings.	Strings, symbols, GPG/libgcrypt fingerprints.	Quantum-vulnerable public-key crypto; migrate away.

Crypto / artefact	Source or config method	Binary/container method	Risk / recommendation note
AES	AES APIs, EVP_aes_*, JCA Cipher("AES/..."), key-size constants.	AES-NI instructions, AES strings, S-box/table constants, library calls.	AES-256 generally acceptable; flag key size/mode issues.
AES-GCM	AES/GCM/NoPadding, AESGCM, EVP_aes_*_gcm; nonce/tag extraction.	GCM symbols, GHASH patterns, cipher suite strings.	Preferred AEAD when nonce handling is correct.
AES-CBC	AES/CBC strings, IV APIs, padding selection.	CBC symbols/strings/wrapper calls.	Allowed only with correct random IV + authentication; recommend GCM/ChaCha20-Poly1305.
AES-CTR	AES/CTR strings and counter APIs.	AES plus counter loop and mode strings.	Needs unique nonce/counter and MAC; prefer AEAD.
AES-ECB	AES/ECB or raw block encryption without mode.	Mode strings or repeated block primitive use.	High risk pattern leakage. Replace with AEAD.
DES	DES APIs and cipher strings.	DES symbols/S-box/permutation constants.	Deprecated/weak. Replace with AES-GCM/ChaCha20-Poly1305.
3DES / TDEA	DESede, TripleDES, EVP_des_ede3.	Symbols/strings/DES constants.	Legacy and slow. Replace with AES-GCM.
RC4 / ARC4	RC4/ARC4 APIs and TLS ciphersuites.	Symbols/strings or 256-byte KSA/PRGA pattern.	Broken stream cipher. Remove.
ChaCha20	ChaCha20 APIs, libsodium, OpenSSL EVP_chacha20.	"expand 32-byte k" constant, symbols, library calls.	Good modern stream primitive; prefer AEAD variant.
ChaCha20-Poly1305	AEAD API names, TLS ciphersuite strings.	ChaCha + Poly1305 signatures/symbols.	Good AEAD, especially on non-AES-NI devices.
XChaCha20-Poly1305	libsodium xchacha20poly1305 APIs.	libsodium symbols/strings.	Good AEAD with larger nonce; track library support.

Crypto / artefact	Source or config method	Binary/container method	Risk / recommendation note
Blowfish / Twofish	Algorithm strings and legacy library APIs.	P-array/S-box constants or symbols.	Usually legacy. Prefer AES-GCM/ChaCha20-Poly1305.
Camellia / IDEA / SM4	Algorithm strings, OpenSSL EVP names, regional crypto configs.	OpenSSL symbols/constants.	Context-specific. Track compliance; not PQC issue directly.

Crypto / artefact	Source or config method	Binary/container method	Risk / recommendation note
MD5	MessageDigest/hashlib/OpenSSL MD5 APIs; checksum vs security use classification.	MD5 symbols/strings/IV and round constants.	Broken for collision resistance. Replace with SHA-256/384 or SHA-3.
SHA-1	SHA1 APIs, certificate signatures, Git-like non-security contexts marked separately.	SHA-1 symbols/constants/certs.	Broken for collision resistance. Replace with SHA-256/384 or SHA-3.
SHA-224 / SHA-256	SHA2 APIs and strings.	Symbols/round constants.	Generally acceptable; SHA-256 minimum for most.
SHA-384 / SHA-512	SHA2-512 family API names and constants.	SHA-512-family constants/symbols.	Good for high-security hashing/signature contexts.
SHA-3 / Keccak	SHA3_* and Keccak APIs.	Keccak permutation constants/strings.	Good modern option; verify protocol compatibility.
BLAKE2 / BLAKE3	hashlib.blake2*, blake3 package, Rust/Go libs.	Symbols/constants/static library fingerprints.	Good hash family; not always compliance default.
RIPEMD-160	RIPEMD160/RMD160 APIs, blockchain usage.	Symbols/round constants.	Legacy/niche; avoid for new security designs.
HMAC	HmacSHA256, hmac.new, HMAC APIs; extract underlying hash.	HMAC symbols or ipad/opad construction with hash.	Good if underlying hash is strong and key managed.
CMAC / GMAC	CMAC/GMAC API names, AES-CMAC configs.	AES + CMAC/GMAC symbols.	Valid MACs; context-specific.
Poly1305	Poly1305 APIs/libsodium.	Symbols/constants.	Good MAC, commonly with ChaCha20.
HKDF	HKDF APIs, extract+expand calls, hash parameter.	EVP_KDF/HMAC patterns and strings.	Good KDF when parameters are strong.
PBKDF2	PBKDF2 APIs; extract hash, salt, iteration count.	PKCS5_PBKDF2_HMAC symbols/strings.	Flag weak iteration/salt. For password storage prefer Argon2id/scrypt/bcrypt where suitable.
bcrypt / scrypt / Argon2	Package imports, encoded prefixes $2b$, $scrypt$, $argon2id$.	Symbols/strings/password hash prefixes.	Good password hashing when parameters are strong.
CRC/checksums	crc32/adler32 APIs.	Symbols/table constants.	Classify as non-cryptographic to avoid false risk claims.

PQC / hybrid crypto	Source or config method	Binary/container method	Risk / recommendation note
ML-KEM / Kyber	ML-KEM, MLKEM, Kyber, liboqs, oqs-provider, PQClean identifiers.	Symbols/strings/OQS provider, static library fingerprints.	Preferred NIST KEM family for PQ key establishment.
ML-DSA / Dilithium	ML-DSA, MLDSA, Dilithium, liboqs/PQClean APIs.	Symbols/strings/OQS/PQClean fingerprints.	Primary NIST PQ signature family.
SLH-DSA / SPHINCS+	SLH-DSA, SPHINCS+, hash-based signature API strings.	Symbols/strings/PQClean/OQS fingerprints.	Conservative hash-based signatures; large signature/performance tradeoffs.
FN-DSA / Falcon	Falcon/FN-DSA identifiers and libraries.	Symbols/strings/static fingerprints.	Track as emerging/optional depending on standards and library support.
HQC	HQC identifiers in OQS/PQClean.	Symbols/strings/static fingerprints.	Track as PQ candidate/standardization related where deployed.
Hybrid key exchange	X25519+ML-KEM, ECDHE+Kyber, TLS hybrid group names.	TLS/OpenSSL provider strings; both classical and PQC call paths.	Best migration bridge when ecosystem supports it.
Hybrid signatures	ECDSA+ML-DSA or RSA+ML-DSA policy/config pairs.	Certificate chain/signature algorithm evidence.	Useful transition strategy, but certificate size and compatibility matter.

Protocol / artefact	Source or config method	Binary/container method	Risk / recommendation note
TLS/SSL	SSLContext, TLS version constants, nginx/apache/envoy configs, cipher suites.	OpenSSL/NSS/wolfSSL symbols, embedded ciphersuite strings, live SSLyze probe.	Disable SSL/TLS 1.0/1.1; prefer TLS 1.3 and PQ/hybrid-ready stacks.
SSH	libssh/paramiko/Go x/crypto ssh APIs, sshd_config algorithms.	libssh/libssh2 symbols, SSH algorithm strings.	Inventory KEX/signature algorithms; plan PQ/hybrid SSH where supported.
IPsec/IKE/VPN	strongSwan/libreswan configs, IKE groups, certs.	IKE/IPsec library strings/symbols.	Quantum-vulnerable DH/ECDH/RSA/ECDSA must be prioritized for long-lived secrets.
X.509 certificates	Parse PEM/DER; extract subject, issuer, key algorithm, key size, signature, validity.	Scan binary/resources for DER/PEM blobs and certificate OIDs.	RSA/ECDSA certs are PQ risk; prioritize public-facing and long-validity chains.
Public/private keys	PEM/DER/JWK/SSH key parsers; Gitleaks/TruffleHog redacted evidence.	ASN.1/DER/PEM headers, OIDs, key structures in binaries/images.	Do not store secrets. Store fingerprint, type, key size, path and risk.
PKCS#12/PFX	File extension plus PKCS#12 APIs and parser.	Magic/ASN.1 structure in files/images.	Extract cert/key metadata, never raw secret.
Java JKS/Keystore	KeyStore("JKS"/"PKCS12"), file detection.	JKS magic/header, Java resource discovery.	Link keys/certs to Java services.
PKCS#11 / HSM	PKCS11 config, C_* APIs, HSM provider names.	Imports/symbols from PKCS#11 shared library.	High-value artefact; risk depends on algorithm and protected operation.
Cloud KMS / Key Vault	AWS/GCP/Azure SDK calls, Terraform/IaC resources, policy configs.	SDK packages/endpoints in containers/binaries.	Inventory key specs, usage and service link. Innovation surface.
Crypto libraries	Imports/dependencies for OpenSSL, BouncyCastle, libsodium, Tink, Botan, wolfSSL, mbedTLS.	Linked DLL/SO names, static library signatures, package inventory.	Mark as Provides until usage evidence is found.

8. Quantum risk engine

The risk engine should not simply label RSA as red and AES as green. It should prioritize systems using a Mosca-style model plus exposure, data lifetime and migration complexity.

```
Inputs:
  X = data confidentiality lifetime
  Y = migration time / replacement lead time
  Z = expected timeline to cryptographically relevant quantum computer
  Q = quantum vulnerability of algorithm family
  E = exposure: internet-facing, internal, offline, cloud, third-party
  B = business criticality / regulatory sensitivity
  C = confidence that the artefact is actually used

Mosca trigger:
  if X + Y > Z -> urgent quantum migration risk

Risk score:
  score = f(Q, X, Y, Z, E, B, C, key_size, protocol, asset_owner, environment)
```

Risk class	Condition	Example action
Critical	Quantum-vulnerable public-key crypto + long-lived sensitive data + high exposure/criticality.	Immediate migration plan; hybrid PQC pilot; executive report.
High	RSA/ECC/DH in production services, signing, VPN, cloud KMS, customer/internal sensitive data.	Prioritize in next release cycle; track owner and blockers.
Medium	Quantum-vulnerable but internal, short-lived, low criticality or uncertain usage.	Confirm usage; replace during planned modernization.
Low	Symmetric AES-256/SHA-256/HMAC with strong parameters or PQC already used.	Monitor and keep inventory.
Informational	Provided by dependency but no use evidence, test fixture, dead code or checksum-only use.	Suppress or mark ProvidedOnly to avoid noisy reports.

8.1 Required report fields

Field	Description
Crypto asset	Algorithm, mode, padding, key size, curve, certificate/signature, protocol version and library/provider version.
Location	File/line/function, binary offset/symbol, image digest/layer, config path, endpoint or cloud resource ID.
Evidence	Detector names, signal types, redacted snippet, confidence, usage state and duplicate links.
Context	Owner, environment, exposure, business criticality, data lifetime and migration estimate.
Risk	Quantum risk, classical weakness, compliance weakness, Mosca result and priority.
Recommendation	Target PQC/hybrid/classical replacement, migration path, compatibility notes and expected cost/latency impact class.

9. PQC and crypto recommendation engine

Recommendations must be deterministic and explainable. They should be generated from a policy registry, not from an LLM. An LLM can rewrite the explanation, but the selected replacement must come from rules and standards.

Detected use	Risk	Suggested path	Decision notes
RSA key exchange / encryption	Quantum-vulnerable	ML-KEM or hybrid classical+ML-KEM	Prefer hybrid during transition; choose parameter level by criticality and protocol support.
RSA signatures	Quantum-vulnerable	ML-DSA or SLH-DSA; hybrid signature if required	Certificate/PKI compatibility is the main blocker.
ECDH / X25519 / DH	Quantum-vulnerable	Hybrid X25519+ML-KEM / ECDHE+ML-KEM where available	Great demo item for TLS migration roadmap.
ECDSA / EdDSA	Quantum-vulnerable	ML-DSA primary; SLH-DSA for conservative/hash-based profile	Track signature size and verification cost.
TLS 1.0/1.1 or weak ciphers	Classical weakness	TLS 1.3, AEAD ciphers, modern groups, PQ hybrid readiness	Fix classical risk before claiming PQ readiness.
AES-128 protecting long-lived sensitive data	Reduced quantum margin	AES-256-GCM or ChaCha20-Poly1305 depending platform	Grover-style speedup motivates larger symmetric margin.
AES-CBC/CTR without authentication	Misuse risk	AES-GCM or ChaCha20-Poly1305	AEAD simplifies secure design.
AES-ECB	High classical weakness	AES-GCM/ChaCha20-Poly1305	Never acceptable for sensitive structured data.
DES/3DES/RC4	Broken/legacy	AES-GCM or ChaCha20-Poly1305	Immediate modernization recommendation.
MD5/SHA-1 security use	Broken collision resistance	SHA-256/SHA-384/SHA-3	Distinguish checksum-only from security use.
PBKDF2 weak parameters	Password risk	Argon2id/scrypt/bcrypt or stronger PBKDF2 policy	Use compliance/stack constraints.
PQC already present	Potentially good	Verify parameters, implementation maturity and hybrid fallback	Do not blindly mark all PQC as safe.

9.1 Recommendation score

```
replacement_score =
  0.35 * risk_reduction
+ 0.20 * standards_alignment
+ 0.15 * compatibility
+ 0.10 * latency_budget_fit
+ 0.10 * migration_effort_fit
+ 0.10 * operational_cost_fit

Output example:
  Finding: ECDHE_RSA TLS endpoint with RSA-2048 certificate
  Priority: High
  Recommendation: migrate service to TLS 1.3, then pilot hybrid X25519+ML-KEM and PQ-capable cert strategy.
  Reason: internet-facing + quantum-vulnerable public-key auth + long-lived sensitive data.
```

10. CLI and GUI

The GUI satisfies the PS. The CLI makes the tool feel enterprise-grade, benchmarkable and CI/CD-ready. Both must call the same core engine.

```
# source, binary, container and cloud scans
ecdat scan ./repo --out scan.ecdat.json --cbom cbom.json
ecdat scan-binary ./bin/app.exe --deep auto --out binary.ecdat.json
ecdat scan-container ghcr.io/org/backend:latest --out image.ecdat.json
ecdat scan-cloud aws --profile readonly --services kms,acm,elb --out cloud.ecdat.json

# risk, recommendations and reports
ecdat risk scan.ecdat.json --profile high-security --crqc-years 10 --out risk.json
ecdat recommend risk.json --target pqc-hybrid --out migration-plan.json
ecdat report scan.ecdat.json --format pdf --out report.pdf

# CI mode
ecdat ci ./repo --fail-on critical --sarif ecdat.sarif --summary-md summary.md
```

GUI screen	Purpose
Target selection	Repo/binary/container/cloud/runtime input cards; user can provide one or many.
Scan plan preview	Shows detectors that will run, expected depth, privacy/redaction, cache hits and estimated cost/time.
Live scan monitor	Progress by surface: source, binary, container, certificates, configs, cloud, runtime.
Crypto inventory	Search/filter all assets by algorithm, key size, risk, service, owner, environment and confidence.
Crypto graph	Visual links: service -> container -> library -> crypto operation -> key/cert -> endpoint/cloud resource.
Risk dashboard	Critical/high/medium/low, Mosca risk, HNDL risk, internet-facing assets and weak classical crypto.
Finding detail	Evidence timeline, source snippet redacted, binary signals, config/runtime agreement and confidence formula.
Migration planner	Recommended PQC/hybrid replacement, blockers, owners, effort, latency/cost class and exportable roadmap.
Reports	CycloneDX CBOM, PDF/HTML executive report, JSON, SARIF for GitHub code scanning.

11. Data storage and output formats

Use append-only scan records and a normalized database. For SIH, SQLite is enough. For enterprise, the graph can move to Neo4j or a property-graph backend, but do not make that a demo dependency.

Table / object	Important fields
scan_run	run_id, target, started_at, git_commit/image_digest, detector_versions, settings, status.
raw_evidence	run_id, detector, file/resource, range, redacted_snippet, signal_type, confidence, hash.
crypto_asset	asset_id, canonical_name, family, algorithm, mode, key_size, curve, protocol, version.
crypto_operation	operation_id, operation_type, asset_ids, location, implementation, usage_state, confidence.
graph_edge	from_id, to_id, relation: uses/provides/implements/configures/protects/exposes/runs_in.
risk_record	operation_id/system_id, mosca_inputs, score, class, explanation, assumptions.
recommendation	finding_id, target_algorithms, migration_pattern, compatibility, effort, latency/cost class.

11.1 Export targets

Export	Use
CycloneDX CBOM JSON/XML	Standard cryptographic inventory deliverable.
ECDAT canonical JSON	Full internal evidence, graph and risk data.
SARIF	CI/CD findings inside GitHub/GitLab security views.
PDF executive report	Judges, management, audit and presentation.
HTML dashboard export	Offline browsing of scan results.
CSV	Simple spreadsheet review for asset owners.

12. Benchmark plan to prove superiority

To beat existing solutions, do not only claim better architecture. Build a benchmark harness that measures coverage, precision, speed and graph quality.

Benchmark	Dataset design	Metric
Source coverage	Synthetic + real repos across Java, Python, JS/TS, Go, C#, Rust, C/C++; seeded crypto wrappers and configs.	Precision/recall per algorithm, parameter extraction rate, wrapper detection rate.
Binary coverage	Compiled samples with dynamic linking, static linking, stripped symbols and embedded constants.	Detection recall, false positives, time, confidence quality.
Container coverage	Images with OS packages, app deps, certs, keys, configs and binaries across repeated layers.	Layer-cache speedup, duplicate suppression, linked evidence count.
Runtime agreement	Local TLS endpoints with known configs and cert chains.	Static vs runtime agreement, negotiated protocol/cipher accuracy.
Graph reconstruction	Services where source, config, cert and container all refer to the same crypto system.	Raw findings reduced to correct operations/systems; duplicate reduction.
Recommendation quality	Known risk scenarios: HNDL, short-lived internal data, weak legacy crypto, PQC already used.	Correct priority and migration recommendation.
Performance	Run on weak laptop profile and CI container.	Files/sec, MB/sec, peak RSS, cache hit speedup, expensive-analysis rate.

12.1 Public comparison targets

Baseline	What to compare
grep/ripgrep rules	Raw string detection baseline.
CBOMkit / Sonar crypto	Source CBOM baseline for supported languages.
cdxgen --include-crypto	BOM/CBOM generation and source/container baseline.
Syft/Trivy	Dependency and container package inventory baseline.
strings + objdump	Cheap binary baseline.
LIEF-only/YARA-only	Ablation: prove evidence fusion is stronger than one binary signal.
ECDAT full pipeline	Expected winner: higher linked coverage, lower false positives, better risk prioritization.

13. Implementation sequence

Build in this order. Do not start with the GUI. The core evidence model and CLI make everything testable.

Phase	Build	Exit criterion
1. Core schema	CIR models, signature registry, canonical names, SQLite store, CLI skeleton.	Can ingest a fake finding and export canonical JSON.
2. Source MVP	CBOMkit/cdxgen integration + Semgrep/Tree-sitter rules for Java, Python, JS/TS, Go, C#.	Detect Tier-1 algorithms with file/line/mode/key-size where available.
3. Dependency/library	Syft/cdxgen package scans + package-to-crypto capability mapping.	Distinguish ProvidedOnly from used evidence.
4. Cert/key/config	PEM/DER/JKS/P12/SSH/JWK parsing, Gitleaks/TruffleHog redacted evidence, nginx/OpenSSL/TLS config rules.	Inventory keys/certs without leaking secrets.
5. Binary MVP	LIEF + strings + YARA-X crypto constants for ELF/PE/Mach-O.	Detect OpenSSL/CNG/libsodium and common algorithms in stripped/simple binaries.
6. Container MVP	Scan Docker/OCI image rootfs; layer cache; run dependency, binary, cert and config scanners inside.	Container scan creates linked CBOM evidence.
7. Evidence fusion	Dedup, confidence scoring, source/binary/container correlation, operation builder.	Raw hits become clear Confirmed/Probable/Possible/ProvidedOnly operations.
8. Risk + recommender	Mosca risk, classical weak crypto rules, PQC/hybrid recommendation registry.	Every finding gets risk class, reason and migration suggestion.
9. GUI	Tauri/React dashboard over core API.	All PS workflows visible: scan, inventory, risk, recommendations, reports.
10. Benchmarks/docs	Benchmark harness, comparison baselines, final PDF/HTML report.	Reproducible numbers for SIH pitch.

13.1 10-day practical target

A realistic SIH prototype should not attempt perfect whole-enterprise cloud coverage. Build source + dependency + container + certificate + basic binary + risk/recommendation extremely well, then add runtime/cloud as innovation modules with at least one working provider demo.

14. What makes this solution stronger than common entries

Common SIH entry	ECDAT stronger version
Regex detects RSA/AES strings.	Language-aware detection plus parameter extraction and source/binary/config corroboration.
Lists every OpenSSL algorithm as used.	Separates Provides from Confirmed use and suppresses dependency-only noise.
Dashboard has many red boxes.	Graph reconstructs actual crypto systems and assigns owner/environment/exposure.
Recommends PQC generically.	Maps operation type to ML-KEM/ML-DSA/SLH-DSA/hybrid strategy with compatibility and cost/latency class.
Scans one repo slowly every time.	Incremental cache, layer digest cache and staged escalation.
No binary support or only strings.	LIEF + YARA-X + capa/FLOSS escalation + optional Ghidra for high-value binaries.
No proof of accuracy.	Benchmark harness with seeded ground truth and public baseline comparisons.

14.1 Final product thesis

ECDAT reconstructs cryptographic systems, not just crypto keywords. The platform converts scattered source, binary, container, config, certificate, runtime and cloud evidence into a single knowledge graph, then turns that graph into a CBOM, quantum risk score and migration roadmap.

15. References and implementation sources

ID	Source	URL / note
R1	SIH26164 problem statement	User-provided problem statement text in chat.
R2	CycloneDX CBOM capability	https://cyclonedx.org/capabilities/cbom/
R3	Authoritative Guide to CBOM	https://cyclonedx.org/guides/OWASP_CycloneDX-Authoritative-Guide-to-CBOM-en.pdf
R4	NIST PQC standards announcement	https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards
R5	FIPS 203 ML-KEM	https://csrc.nist.gov/pubs/fips/203/final
R6	FIPS 204 ML-DSA	https://csrc.nist.gov/pubs/fips/204/final
R7	FIPS 205 SLH-DSA	https://csrc.nist.gov/pubs/fips/205/final
R8	NIST NCCoE Migration to PQC	https://www.nccoe.nist.gov/applied-cryptography/migration-to-pqc
R9	NIST SP 1800-38 PQC migration / cryptographic discovery	https://www.nccoe.nist.gov/publications/practice-guide/migration-post-quantum-cryptography-nist-sp-1800-38-practice-guide
R10	Cryptoscope paper	https://arxiv.org/abs/2503.19531
R11	CryptoGuard paper	https://arxiv.org/abs/1806.06881
R12	Code Property Graph / Joern docs	https://docs.joern.io/code-property-graph/
R13	CBOMkit	https://github.com/cbomkit/cbomkit
R14	CBOMkit-theia	https://github.com/cbomkit/cbomkit-theia
R15	cdxgen	https://github.com/cdxgen/cdxgen
R16	Syft	https://github.com/anchore/syft
R17	Semgrep	https://github.com/semgrep/semgrep
R18	Tree-sitter	https://github.com/tree-sitter/tree-sitter
R19	LIEF	https://github.com/lief-project/LIEF
R20	YARA-X	https://github.com/VirusTotal/yara-x
R21	Mandiant capa	https://github.com/mandiant/capa
R22	Mandiant FLOSS	https://github.com/mandiant/flare-floss
R23	Ghidra	https://github.com/NationalSecurityAgency/ghidra
R24	Trivy	https://github.com/aquasecurity/trivy
R25	Gitleaks	https://github.com/gitleaks/gitleaks
R26	TruffleHog	https://github.com/trufflesecurity/trufflehog
R27	SSLyze	https://github.com/nabla-c0d3/sslyze
R28	testssl.sh	https://github.com/testssl/testssl.sh
R29	Open Quantum Safe / liboqs	https://openquantumsafe.org/liboqs/